Yes, this is a functional distributed object mechanism—but it is written in a few hundred lines of Ruby code. No C, nothing fancy, just plain old Ruby code. Of course, it has no naming service, trader service, or anything like you'd see in a full-fledged distributed system, but it is simple and faster than you might think.

What's Next

The important thing to remember about Ruby is that there isn't a big difference between "compile time" and "runtime." It's all the same. You can add code to a running process. You can redefine methods on the fly, change their scope from public to private, and so on. You can even alter basic types, such as `Class` and `Object`. Once you get used to this flexibility, it is hard to go back to a static language such as C++ or even to a half-static language such as Java.

But then, why would you want to do that?

Now its time to take a look at Ruby's syntax in a more structured way.